

@devopschallengehub



Linux for DevOps – Ultimate Short Guide

How to study ?

1. Hands-on > Reading (80% retention)

You remember Linux commands only **when your hands use them**.

Do this:

- Take a small EC2 instance on AWS cloud (create free cloud account) or local VM.
- Practice daily tasks: create users, mount disks, rotate logs, debug services.
- Break things → fix them → memory becomes permanent.

Rule:

👉 *If your hands didn't type it, your brain won't remember it.*

2. Visualize Concepts (story-based memory)

Complex commands like `chown`, `chgrp`, permissions, mount, SSH keys can be turned into stories.

Example:

- `chown` = "Change **owner** of the house."
- `chgrp` = "Change the **group** living in the house."
- `chmod` = "Change the rules of the house."

Your mind remembers images, not text.

3. Use High-Level Patterns (reduce complexity)

Instead of memorizing 50 commands, learn the pattern:

User management pattern

`adduser` → create

`usermod` → modify

`chown` → change owner

`chgrp` → change group

`chmod` → give permissions

Disk pattern

`lsblk` → view disk

`fdisk` → partition

`mkfs` → format

`mount` → attach

fstab → make permanent

Your brain remembers flows, not commands.

4. Flashcards (15 minutes/day) during tea break , or in the office bus, revise .

Create simple cards like:

Q: chown ubuntu:dev team.txt

A: Change owner to ubuntu, group to dev.

Brains love repetition.

Spaced repetition (Anki) = **automatic long-term memory. (After 1 day, 3, 7, 14, 30, 90 days)**

5. Random Question Drills (self-quizzing)

Ask yourself questions:

- “How do I give group read-only permission?”
- “How do I mount a new disk?”
- “How do I rotate SSH keys?”
- “Why does ./deploy.sh fail when bash deploy.sh works?”

This builds **real-world recall**.

6. Create Your Own “Mini SOPs” (Standard Operating Procedures)

Every time you solve something, write a short SOP:

Fix: Disk 100% full

1. df -h
2. du -sh /*
3. truncate logs
4. docker prune

Your own notes become a personal DevOps playbook.

7. Practice 2-3 Scenario Daily

Pick 1 scenario per day (10–15 minutes):

- SSH failure
- Cron not running
- Memory leak
- Mount disk
- Permission denied
- Systemd crash loop

Solve it like a real on-call engineer.

This builds muscle memory + confidence.

8. Use Mind Maps (great for complex topics)

Topics like permissions, file systems, systemd, networking → draw a simple mind map.

Your brain sees the *structure*, not scattered commands.

9. Build Real Mini Projects

Examples:

- Create a backup script + cron
- Setup log rotation
- Configure systemd service for your app

- Setup SSH key rotation
- Create read-only users for auditors
- Mount new EBS volume

Projects give you deep understanding.

1 0 Final Formula to Master Linux (The 20/80 Rule)

20% of concepts → solve 80% of real DevOps problems

1. Users & permissions
2. Disk & filesystem
3. Systemd services
4. Networking basics
5. Logs & troubleshooting
6. SSH & key management
7. Packages & updates
8. Cron
9. Processes & memory
10. Commands: awk, grep, sed, tail, find

Master these → you're better than 90% of candidates.

Quick Summary

1 File System Basics

What it is: Linux stores everything in a hierarchical tree structure.

Important paths:

- /home → users' files
- /var/log → logs
- /etc → config files
- /opt → optional apps
- /usr/bin → commands

Example:

```
cd /var/log
```

```
ls -l
```

👉 Used daily when checking logs in DevOps.

2 File Permissions & Ownership

What it is: Controls who can read/write/execute files.

Format: rwx for **owner**, **group**, **others**.

Example:

```
chmod 755 script.sh # Owner full, others can read+execute
```

```
chown ubuntu:ubuntu file.txt
```

👉 Useful for fixing "permission denied" in servers.

3 Process Management

What it is: Manage running programs.

Commands:

- `ps aux` → show all processes
- `top` / `htop` → real-time view
- `kill -9 PID` → force stop

Example:

`ps aux | grep node`

`kill -9 3456`

👉 Used when apps get stuck or consume too much CPU.

4 System Monitoring

Check resource usage:

`top` # CPU + memory

`free -h` # RAM usage

`df -h` # disk

`du -sh .` # folder size

`uptime` # server load

👉 In DevOps, used during incidents or performance checks.

5 Networking

Key commands:

- `ip a` → view network interfaces
- `ping google.com` → check connectivity
- `curl URL` → check API/website, **cURL as the "Universal API Client"**
- `netstat -tulnp` → check open ports
- `ss -ltn` → modern replacement

netstat -tulnp

- Shows listening TCP/UDP ports with PIDs/process names
- `-t`: TCP ports
- `-u`: UDP ports
- `-l`: Listening ports only
- `-n`: Show numerical addresses (no DNS resolution)
- `-p`: Show process ID/name

ss -ltn

- Modern replacement for netstat (faster, more info)
- `-l`: Listening sockets only
- `-t`: TCP ports
- `-n`: Numerical addresses

Key difference: `ss` is faster and shows more detailed socket information, while `netstat` is more universally available but older. For most port checking tasks, `ss` is preferred.

Example:

`curl http://localhost:3000/health`

👉 Used to debug microservices.

6 Package Management

Installing software using package managers:

For Ubuntu:

`sudo apt update`

`sudo apt install nginx`

👉 Required for installing tools like Git, AWS CLI, Docker.

7 Service Management (systemd)

Start/stop/restart services:

`sudo systemctl status nginx`

`sudo systemctl start nginx`

`sudo systemctl enable nginx`

👉 Used constantly in servers (e.g., start Docker, restart backend apps).

8 Users & Groups

1. groupadd – Create a New Group

Used to create a Linux group (developers, testers, backup, readers, etc.)

Example

`sudo groupadd readers`

This creates a group named **readers**.

Useful when you want multiple users to have the same permissions on a folder or file.

2. adduser (Debian/Ubuntu) / useradd (RHEL/CentOS) – Add a New User

adduser

- High-level, user-friendly
- Creates home folder
- Prompts for password automatically

`sudo adduser devuser`

useradd

- Low-level
- Does NOT create home directory by default unless you add flags

`sudo useradd -m devuser`

3. usermod – Modify Existing User

Used to update user details like:

- Add user to a group
- Change home folder
- Change shell
- Lock account
- Grant sudo privileges

Example: Add user to a group

`sudo usermod -aG readers devuser`

-aG = append to group (do not remove existing groups)

Example: Lock a user's login

`sudo usermod --lock devuser`

Example: Change shell

```
sudo usermod -s /bin/bash devuser
```

4. chgrp – Change Group Ownership

Only changes the **group** of a file/folder (NOT the owner).

Example

```
sudo chgrp readers /var/log/app.log
```

Now the file belongs to group **readers**.

5. chown – Change Owner or Owner+Group

Example: Change owner

```
sudo chown ubuntu file.txt
```

Example: Change owner + group

```
sudo chown ubuntu:readers file.txt
```

6. chmod – Change File/Folder Permissions

Controls **read (r)**, **write (w)**, **execute (x)** permissions.

Example

r = 4

w = 2

x = 1

Make a script executable

```
chmod +x deploy.sh
```

Interview question Give read-write to owner, read-only to group and others

```
chmod 644 file.txt
```

Example

Create/manage users:

```
sudo adduser dev      # Creates a new user 'dev'
```

```
sudo usermod -aG docker dev # Adds 'dev' to docker group
```

Purpose: Allows the 'dev' user to run Docker commands without sudo, enabling container management in CI/CD pipelines and development environments without root privileges.

Security note: This gives the user significant privileges (equivalent to root in containers), so use carefully in production.

👉 Also used to give limited access to team members/automation users.

9 SSH & Keys

Access remote servers:

```
ssh ubuntu@3.110.22.50
```

Generate SSH key:

```
ssh-keygen
```

👉 Everyday DevOps work = SSH into servers.

```
ssh -i key.pem ubuntu@IP_ADDRESS
```

This means:

ssh → connect to a remote Linux server

-i key.pem → use this private key for authentication

ubuntu → username on the remote server

@IP_ADDRESS → server's public IPv4 address

Example

```
ssh -i prod-server.pem ubuntu@3.109.54.12
```

⚠ Common Mistakes to Avoid

❌ Mistake: Wrong file permissions

If your key is world-readable, SSH will reject it:

Permissions 0644 for 'key.pem' are too open.

✓ Fix:

```
chmod 400 key.pem
```

10 Environment Variables

Temporary:

```
export PORT=8080
```

Permanent: add to ~/.bashrc.

Check:

```
echo $PORT
```

👉 Used in apps, pipelines, Docker, CI/CD.

11. Logs & Log Analysis

Common logs under /var/log/.

```
tail -f /var/log/syslog # Live stream of system logs in real time
```

```
tail -n 100 app.log # Show the last 100 lines of the application log
```

```
grep "ERROR" app.log # Search for ERROR entries inside the log file
```

👉 Debug production issues quickly

12. Storage & Disk

Mount points, disk usage:

```
df -h # total disk
```

```
du -sh /var/log # folder usage
```

👉 Useful when server becomes full.

13. Archiving & Compression

```
tar -czvf backup.tar.gz /folder
```

```
tar -xzvf backup.tar.gz
```

👉 Used for backups and transferring code.

14 Crontab (Scheduling Jobs)

Run scripts on schedule:

`crontab -e`

Run every day at 2am

`0 2 * * * /home/ubuntu/backup.sh`

👉 Great for automation.

* * * * *

| | | | |

| | | | — Day of Week (0–6) Sun=0 or 7

| | | — Month (1–12)

| | — Day of Month (1–31)

| — Hour (0–23)

— Minute (0–59)

`0 2 * * * /home/ubuntu/backup.sh`

Field-by-field explanation:

0 → Minute

Run at 0th minute

(i.e., at :00)

2 → Hour

Run at 2 AM

* → Day of Month

Any day (1–31)

* → Month

Every month

* → Day of Week

Every day (Mon–Sun)

Final Meaning

Runs the script `/home/ubuntu/backup.sh` every day at 2:00 AM.

15. Sudo & Root Access

Run command as admin:

`sudo su -`

`sudo -i`

👉 Needed for installations and server maintenance.

Both commands are used to switch to the **root user**, but with slightly different behavior.

Run `su -` **with sudo**, which switches to **root user** and loads **root's full login environment**.

(`sudo su -`)

Start a **root login shell** directly using `sudo`. (`sudo -i`)

16 File Editing with vim/nano

Basic Nano:

`nano config.txt`

Basic Vim:

- **insert mode:** `i`, exit with `Esc`
- **save & quit:** `:wq`
- **undo/redo:** `u` / `Ctrl+r`
- **delete line:** `dd`
- **copy/paste:** `yy`, `p`
- **search:** `/pattern`
- 👉 You will edit configs daily.

17 Shell Scripting

Automate tasks.

Simple script:

`#!/bin/bash`

`echo "Deploying app..."`

`npm install`

`systemctl restart app`

Make executable:

`chmod +x deploy.sh`

👉 Used in CI/CD, automation, DevOps pipelines.

18 cURL & wget

Test APIs or download files.

`curl http://localhost:8080/health`

`wget https://nodejs.org/package.tar.gz`

👉 Used in troubleshooting and automation.

19 System Info

`uname -a` # Show complete system info: kernel name, version, architecture

`lsb_release -a` # Show Linux distribution details (Ubuntu version, codename, release)

`hostnamectl` # Show or change system hostname and display OS + kernel info

👉 Know OS version, helpful in debugging.

20 Disk Partitioning & Mount

Basics only:

```
mount /dev/sdb1 /data
```

```
umount /data
```

👉 Needed when attaching new storage (AWS EBS) to your EC2 instance. You may want to add additional EBS storage volume of say 100GB.

File System & Disk Management — Most Common Commands

```
lsblk          # List all block devices (disks, partitions)
```

```
fdisk -l       # Show disk partitions and details
```

```
df -h          # Show mounted filesystems and disk usage in human-readable form
```

```
du -sh *       # Show size of files/folders in current directory
```

Create Filesystem / Format Disk (mkfs)

```
sudo mkfs.ext4 /dev/sdb1  # Format /dev/sdb1 partition with EXT4 filesystem
```

```
sudo mkfs.xfs /dev/sdb1   # Format partition with XFS filesystem
```

```
sudo mkfs.vfat /dev/sdb1  # Format as FAT32 (USB drives)
```

Warning: This **erases data** on the partition.

Mount / Unmount

```
sudo mount /dev/sdb1 /data # Mount partition /dev/sdb1 to folder /data
```

```
sudo umount /data          # Unmount filesystem from /data
```

```
sudo umount /dev/sdb1      # Unmount by device name
```

Create Mount Directory

```
sudo mkdir -p /data        # Create mount directory if not exists
```

Persistent Mount (for reboot)

Edit /etc/fstab to auto-mount on reboot:

```
/dev/sdb1 /data ext4 defaults 0 0 # Auto-mount sdb1 as ext4 in /data at boot
```

Check Filesystem Health

```
sudo fsck /dev/sdb1        # Check and repair filesystem errors on partition
```

View Mounted Devices

```
mount | grep sdb1          # Show where /dev/sdb1 is mounted
```

```
findmnt                  # Display all current mounts in a tree format
```

Quick Partitioning (Interactive)

```
sudo fdisk /dev/sdb        # Create partitions on disk /dev/sdb (MBR style)
```

```
sudo parted /dev/sdb       # Create partitions (GPT friendly)
```

, etc.)

21 Firewalls

Ubuntu:

ufw status

ufw allow 22

ufw allow 80

👉 Used to open service ports.

22 SELinux/AppArmor (RHEL/Ubuntu security)

Disable *temporarily* (for debugging):

setenforce 0

👉 Important on RedHat-based systems.

23 Kernel & System Limits

Check open file limits:

ulimit -n

Set limits in /etc/security/limits.conf.

👉 Needed for high-traffic servers.

Interview Questions

How do you use curl as a DevOps Engineer in your day-to-day life?

1. **Query** the state of systems, clusters, and applications.
2. **Debug** network connectivity and API responses within complex systems.
3. **Trigger** builds, deployments, and jobs.
4. **Configure** infrastructure and services via their API.

API Testing:

curl -X GET <https://api.service.com/health>

Check if a service is responding.

File Operations

curl -o config.yaml <https://config-server.example.com/app/prod/config.yaml>

Download configuration files from remote server

Authentication Methods

curl -u username:password <https://api.example.com>

Basic authentication with username and password

CI/CD Trigger (Start a Jenkins build remotely.)

curl -X POST <https://jenkins.example.com/job/build/build?token=TOKEN>

Monitoring & Debugging

curl -v <https://api.example.com>

Show detailed request/response for debugging connection issues (v for verbose)

Example: Checking an AWS EC2 Instance's Metadata

bash

Get the instance's public IP from its metadata service

curl <http://169.254.169.254/latest/meta-data/public-ipv4>

Get the instance type (e.g., t3.medium)

curl <http://169.254.169.254/latest/meta-data/instance-type>

169.254.169.254 = Special AWS Instance Metadata Service (IMDS)

No internet required. Accessible **only from inside the EC2**.

Example: Using GitHub Actions API to Trigger a Workflow

bash

Trigger a workflow dispatch event

curl -X POST \

-H "Authorization: token \$GITHUB_TOKEN" \

-H "Accept: application/vnd.github.v3+json" \

https://api.github.com/repos/:owner/:repo/actions/workflows/:workflow_id/dispatches \

-d '{"ref": "main"}'

How would you set up a monitoring system where you have created a non-login system user and placed it in a restricted group. Then assign this group read-only permissions on specific directories like logs and backups. (To ensure that our monitoring agents can read what they need but cannot modify anything on the server.)?

Goal

- Create a non-login system user (for Prometheus/node_exporter, Datadog, Nagios, Zabbix, etc.).
- Put the user in a restricted group.
- Allow that group read-only access to only specific directories (like /var/log and /data/backups).
- Ensure monitoring agents can read but never modify.

Short Answer:

I create a system user with no login shell, add it to a restricted group, and then assign that group **read-only** access using chmod or ACLs. I ensure the directories have g+s so new logs

inherit the group. This allows monitoring agents to read logs/backups but guarantees they cannot modify anything.

1 Create a system user with NO login shell

This prevents interactive login and improves security.

```
sudo useradd -r -s /usr/sbin/nologin monitoruser
```

Meaning:

- -r: system account
- -s /usr/sbin/nologin: no shell

2 Create a restricted read-only group

```
sudo groupadd monitorread
```

Add the monitoring user into this group:

```
sudo usermod -aG monitorread monitoruser
```

3 Assign read-only permissions to specific directories

Example directories:

- /var/log/app/
- /data/backups/

A. Change the group ownership

This ensures the group is tied to these directories:

```
sudo chgrp -R monitorread /var/log/app
```

```
sudo chgrp -R monitorread /data/backups
```

B. Give read + execute permissions to the group

For directories, monitoring tools need:

- r (read): view files
- x (execute): enter the directory

```
sudo chmod -R 750 /var/log/app
```

```
sudo chmod -R 750 /data/backups
```

Why 750?

- Owner: read/write
- Group: read/execute (NO write)
- Others: no access

Write access (w) is NOT given → ensures safety.

4 Make sure new log files inherit the correct group

This is a crucial detail in production.

Set the **setgid** bit on directories:

```
sudo chmod g+s /var/log/app
```

```
sudo chmod g+s /data/backups
```

Now **any new log/backups created will automatically belong to monitorread group**, so monitoring user can read them.

5 Optional but common) Use ACLs for finer control

If you don't want to change group ownership, use ACLs:

```
sudo setfacl -R -m g:monitorread:rx /var/log/app
sudo setfacl -R -m g:monitorread:rx /data/backups
To apply default ACLs for future files:
sudo setfacl -R -d -m g:monitorread:rx /var/log/app
sudo setfacl -R -d -m g:monitorread:rx /data/backups
ACL method is useful if logs are owned by multiple applications.
```

6 Verify access

As the monitoring user:

```
sudo -u monitoruser ls /var/log/app
sudo -u monitoruser cat /var/log/app/app.log
```

Ensure it works:

- Can read files
- Cannot modify/delete:

```
sudo -u monitoruser touch /var/log/app/testfile
# should fail: "Permission denied"
```

Final interview answer

I create a system user with no login shell, add it to a restricted group, and then assign that group **read-only** access using chmod or ACLs. I ensure the directories have g+s so new logs inherit the group. This allows monitoring agents to read logs/backups but guarantees they cannot modify anything.

Disk is full on a production server, application is crashing. What will you do ?

Problem:

Your application crashes. Running df -h shows / is 100% full.

Root Cause Patterns:

- Logs consuming space
- A process dumping core files
- Docker overlay size full

How do you troubleshoot?

Answer:

```
df -h          # Check which mount is full
du -sh /*      # Find which top-level folder is heavy
cd /var/log && du -sh *  # Identify large logs
find . -type f -size +50M (Find files larger than 50 MB)
sudo truncate -s 0 app.log # Safely clear a log file
docker system prune -a    # Optional: Clean unused docker images/containers
```

What interviewer looks for:

Structured debugging → *identify mount* → *find heavy folder* → *delete safely*.

Scenario: A new EBS volume is attached, but your app can't use it. What will you do ?

You attached /dev/sdb to your EC2 instance. lsblk shows the disk, but mounting fails.

Correct steps?

Answer:

```
lsblk                # Ensure disk is visible
sudo fdisk /dev/sdb   # Create a partition (n → p → w)
sudo mkfs.ext4 /dev/sdb1 # Format partition
sudo mkdir /data      # Create mount point
sudo mount /dev/sdb1 /data # Mount it
echo "/dev/sdb1 /data ext4 defaults 0 0" | sudo tee -a /etc/fstab # Make it persistent
```

Interviewer checks:

You understand disk → partition → format → mount → fstab flow.

Scenario: SSH access denied even though key is correct. What will you do ?

You try:

```
ssh -i key.pem ubuntu@3.110.45.2
```

But you get:

Permission denied (publickey)

How to fix?

Answer:

Check these:

```
chmod 400 key.pem          # Fix permissions
cat /var/log/auth.log | grep sshd # Check server-side logs
sudo chown ubuntu:ubuntu ~/.ssh/authorized_keys
sudo chmod 600 ~/.ssh/authorized_keys
```

Interviewer looks for:

Permission fix, server logs, .ssh directory hygiene.

Scenario: Cron job not running even though it's correctly set. How will you debug and fix?

Cron entry:

```
0 2 * * * /home/ubuntu/backup.sh
```

Backup didn't run. Why?

Answer:

Common root causes:

```
sudo chmod +x /home/ubuntu/backup.sh # Script not executable
Check shebang line: #!/bin/bash      # Missing interpreter
Absolute paths required in cron      # PATH is different
Redirect output to logs              # Debug
Improved cron:
0 2 * * * /home/ubuntu/backup.sh >> /var/log/backup.log 2>&1
```

Scenario: Application failing due to wrong file permissions. How will you debug and fix?

Your app logs show:

Permission denied: /data/app_logs

Root-cause reasoning

Typical causes:

- Directory owned by root:root and app runs as appuser -> no write.
- Permissions are drwxr-xr-x (755) -> group/others cannot write.
- ACLs override POSIX and deny access.
- SELinux context prevents write despite correct POSIX perms.

- Parent directory permissions block traversal.

Debug

- Don't run recursive fixes blindly on production. Inspect first. Note current ownership and permissions so you can roll back if needed. `ls -ld /data/app_logs`
- Confirm ownership and mode (`stat /data/app_logs` or `ls -ld /data/app_logs`)
- Check which user the app runs as
- Try to reproduce as that user (`sudo -u appuser -- bash -c 'touch /data/app_logs/testfile'` # will fail if no write)

Fix — least-privilege, step-by-step

If the app should run as `appuser:appgroup` and only that app needs access

1. Make `appuser` (owner) of the directory:

```
sudo chown -R appuser:appgroup /data/app_logs
```

Why: `chown` sets owner and group. Making the app user the owner is the most restrictive and clear.

2. Give owner read/write/execute, group maybe read/execute (if other group members need read), others none:

```
sudo chmod -R 750 /data/app_logs
```

7 (owner rwx), 5 (group r-x), 0 (others ---)

Why: 750 allows the owner to write and group to read/execute (enter dir); others cannot access.

3. If only the owner needs write and group needs no write:

```
sudo chmod -R 750 /data/app_logs
```

Interviewer checks: approach, debug , identify ownership, permission, then provide least-privilege

You choose correct command: *chown, chgrp, chmod, usermod*.

Scenario: A server is slow; you suspect high CPU or RAM usage. How will you debug ?

How do you check?

Answer:

```
top          # Real-time CPU/memory
htop         # Better view (if installed)
free -h      # Memory usage summary
iostat -x    # Disk I/O bottlenecks
dmesg | tail # Kernel warnings (OOM-kill)
```

Expected from candidate:

Troubleshoot CPU, IO, memory in order.

Scenario: You made changes in `/etc/fstab` and server won't reboot. How will you fix it ?

Classic problem: incorrect `fstab` → system fails to mount → stuck.

Fix:

Answer:

1. Boot in EC2 recovery (AWS) or single-user mode.
2. Open the root volume.
3. Fix /etc/fstab:
sudo nano /etc/fstab
Remove the wrong line.

Scenario: Log file is too big; you want to rotate it without downtime. What will you do ?

App writes heavily to /var/log/app.log.

Answer:

sudo truncate -s 0 /var/log/app.log

OR enable logrotate:

sudo nano /etc/logrotate.d/app

Scenario: You need to monitor a live log for errors. How do you do it ?

Answer:

```
tail -f /var/log/syslog          # Real-time system logs
tail -n 100 app.log             # Last 100 lines
grep "ERROR" app.log            # Search for errors
tail -f app.log | grep ERROR     # Live error monitoring
```

Scenario: A new user needs read-only access. How do you do that ?

Solution:

```
sudo groupadd readers           # Create a dedicated group for read-only users
sudo usermod -aG readers developer # Add the user 'developer' to the readers group
sudo chgrp -R readers /var/log/app/ # Change group ownership of the log directory to 'readers'
sudo chmod -R g+r /var/log/app/    # Give read-only (r) permission to the group on all files
```

Q: How do you locate and delete all empty files within a specific directory?

A: Using the find command. For example: find /path/to/directory -type f -empty -delete

Q: Can you give an example of a Bash one-liner to delete files older than a certain date?

A: Yes, you can use find with the -mtime flag. For example, to delete files older than 30 days: find /path -type f -mtime +30 -delete

Q: What is the difference between a "Crash" and a "Panic" in Linux?

A: A **crash** typically refers to the failure of a single application or process. A **kernel panic** is a fatal error encountered by the operating system kernel from which it cannot safely recover, causing the entire system to halt.

Q: How do you locate a file on a Linux system?

A: The two primary tools are **find** (powerful, searches in real-time) and **plocate** (or **locate**, which uses a pre-built database for faster searches).

Q. How will you delete all files which are inside /home/ubuntu/VideoFiles/ and file size is more than 600MB?

First, list the files that will be deleted

```
find /home/ubuntu/VideoFiles/ -type f -size +600M -ls
```

If you're satisfied with the list, then delete them

```
find /home/ubuntu/VideoFiles/ -type f -size +600M -delete
```

Using find with confirmation prompt

```
find /home/ubuntu/VideoFiles/ -type f -size +600M -ok rm {} \;
```

Important Notes:

- -type f: Ensures only files (not directories) are selected
- -size +600M: Finds files larger than 600MB
- -delete: Directly deletes the files (use with caution)
- -exec rm {} \;: Alternative method using rm command
- -ok: Safer version that asks for confirmation

Q: What are some key commands for monitoring Linux performance?

A: Common performance tools include:

- uptime - Shows system load averages.
- vmstat - Reports virtual memory statistics.
- iostat - Shows CPU and disk I/O statistics.
- free - Displays amount of free and used memory.
- sar - System Activity Reporter, for collecting and reporting historical performance data.

Scenario: Your Node.js application deployed on an EC2 instance suddenly stopped responding. Users are reporting 502 errors from the load balancer. You need to investigate what went wrong.

Question: Where would you check first and what commands would you run?

Answer:

```
bash
```

Navigate to application logs

```
cd /var/log
```

Check system logs for errors

```
tail -f /var/log/syslog | grep -i error
```

Check if application-specific logs exist

```
ls -l /var/log/nodejs/
```

View last 100 lines of application log

```
tail -n 100 /var/log/nodejs/app.log
```

Search for specific error patterns

```
grep "ECONNREFUSED\|timeout\|crashed" /var/log/nodejs/app.log
```

Scenario: Your CI/CD pipeline fails during deployment with the error: Permission denied: ./deploy.sh. The script exists in /home/ubuntu/scripts/ but won't execute.

Question: What's the issue and how do you fix it?

Answer:

bash

Check current permissions

```
ls -l /home/ubuntu/scripts/deploy.sh
```

Output: -rw-r--r-- 1 ubuntu ubuntu 245 Nov 19 10:30 deploy.sh

Problem: Script lacks execute permission

Fix: Add execute permission

```
chmod +x /home/ubuntu/scripts/deploy.sh
```

Or use numeric notation (recommended for deployment scripts)

```
chmod 755 /home/ubuntu/scripts/deploy.sh
```

Verify

```
ls -l /home/ubuntu/scripts/deploy.sh
```

Output: -rwxr-xr-x 1 ubuntu ubuntu 245 Nov 19 10:30 deploy.sh

Scenario: Your monitoring alerts show CPU usage at 95% on your application server. The application is slow and users are complaining. You need to identify and kill the problematic process.

Question: How do you identify and handle the rogue process?

Answer:

bash

Check real-time CPU usage

```
top
```

Press 'P' to sort by CPU usage

Or use htop for better visualization

```
htop
```

Find specific process consuming resources

```
ps aux | grep node | sort -nrk 3 | head -5
```

ps aux

Lists **all running processes** with details like:

USER, PID, CPU%, MEM%, COMMAND, etc.

grep node

Filters the list to show only processes containing the word **node**

(e.g., Node.js apps).

sort -nrk 3

Sorts the output based on a column:

- -n → numeric sort
- -r → reverse order (largest first)
- -k 3 → sort by **column 3**, which is **CPU usage (%)**

So you're sorting Node.js processes by **highest CPU usage**.

head -5

Shows **top 5** Node.js processes consuming the most CPU.

Final meaning

Show the top 5 Node.js processes using the most CPU.

Example output shows PID 4523 consuming 87% CPU

Check what this process is doing

`ps aux | grep 4523`

Kill the problematic process

`kill -9 4523`

Restart the service properly

`sudo systemctl restart node-app`

Scenario: Your application crashes with error ENOSPC. Deployments are failing and logs aren't being written.

Question: How do you diagnose and resolve the issue?

Answer:

ENOSPC: no space left on device

bash

Check overall disk usage

`df -h`

Output shows /dev/xvda1 is 98% full

Find which directories are consuming space

`du -sh /var/* | sort -hr | head -10`

Discovered /var/log is using 45GB

`du -sh /var/log/*`

Find large log files

`find /var/log -type f -size +100M -exec ls -lh {} \;`

Clean old logs

`sudo journalctl --vacuum-time=7d`

Remove old compressed logs

`sudo rm -f /var/log/*.gz`

```
# Check application logs
du -sh /var/log/nodejs/
# If app logs are huge, truncate or rotate them
sudo truncate -s 0 /var/log/nodejs/app.log
```

```
# Verify space freed
df -h
```

Scenario: You deployed a new API service on port 8080, but when you try to access it from your browser or another server, the connection times out. The application is running fine locally on the server.

Question: What could be blocking the connection and how do you fix it?

Answer:

```
bash
```

```
# First, verify the service is listening
```

```
netstat -tulnp | grep 8080
```

```
# Or modern alternative
```

```
ss -tulnp | grep 8080
```

```
# Test locally on the server
```

```
curl http://localhost:8080/health
```

```
# Works locally ✓
```

```
# Check firewall status
```

```
sudo ufw status
```

```
# Firewall is blocking the port
```

```
# Allow incoming traffic on port 8080
```

```
sudo ufw allow 8080/tcp
```

```
# Reload firewall
```

```
sudo ufw reload
```

```
# Verify the rule is added
```

```
sudo ufw status numbered
```

```
# Also check cloud security group (AWS/Azure/GCP)
```

```
# Add inbound rule for port 8080 from your IP or 0.0.0.0/0
```

```
# Test from external machine
```

```
curl http://external-ip:8080/health
```

Scenario: You're setting up a new Ubuntu server for deployment. When you try to run git clone, you get command not found: git.

Question: How do you install Git and other essential DevOps tools?

Answer:

```
bash
# Update package repository
sudo apt update

# Install Git
sudo apt install git -y

# Verify installation
git --version

# Install other common DevOps tools
sudo apt install curl wget vim htop net-tools -y

# For AWS CLI
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install

# Verify
aws --version
```

Scenario: After making configuration changes to Nginx, you run `sudo systemctl restart nginx` but the service fails to start. Your website is down.

Question: How do you troubleshoot and fix the Nginx service?

Answer:

```
bash
# Check service status
sudo systemctl status nginx
# Shows "failed" status

# View detailed error logs
sudo journalctl -u nginx -n 50

# Test Nginx configuration for syntax errors
sudo nginx -t
# Output: nginx: [emerg] unexpected ";" in /etc/nginx/sites-enabled/myapp:15

# Edit the problematic config file
sudo vim /etc/nginx/sites-enabled/myapp

# Fix the syntax error (remove duplicate semicolon)

# Test configuration again
sudo nginx -t
# Output: nginx: configuration file /etc/nginx/nginx.conf test is successful

# Start the service
```

```
sudo systemctl start nginx
```

```
# Enable it to start on boot  
sudo systemctl enable nginx
```

```
# Verify it's running  
sudo systemctl status nginx  
curl http://localhost
```

Scenario: A new developer joined your team. They can SSH into the server but get permission denied when running docker ps.

Question: How do you give them proper Docker access?

Answer:

Will check if they have to use sudo for every Docker command.

So question is how do you give them proper Docker access without sudo?

One liner answer: I will add developer user to the docker group.

Check if docker group exists

```
getent group docker
```

Add user to docker group

```
sudo usermod -aG docker developer1
```

Verify user is in group

```
groups developer1
```

User must log out and back in for changes to take effect

Or run this (one-time workaround)

```
newgrp docker
```

Now test without sudo

```
docker ps
```

```
docker run hello-world
```

Scenario: You deployed your Node.js application, but it crashes immediately with error: DATABASE_URL is not defined. The app needs environment variables to connect to the database.

Question: How do you set and persist environment variables correctly?

Answer:

```
bash
```

Temporary (current session only) - NOT recommended for production

```
export DATABASE_URL="postgresql://user:pass@localhost:5432/mydb"
```

```
export PORT=8080
```

```
export NODE_ENV=production
```

Verify

```
echo $DATABASE_URL
```

Permanent solution 1: Add to user's .bashrc

```
nano ~/.bashrc
```

Add at the end:

```
export DATABASE_URL="postgresql://user:pass@localhost:5432/mydb"
```

```
export PORT=8080
```

Reload

```
source ~/.bashrc
```

Permanent solution 2: System-wide environment file

```
sudo nano /etc/environment
```

Add:

```
DATABASE_URL="postgresql://user:pass@localhost:5432/mydb"
```

Best practice for production: Use .env file with application

```
cd /home/ubuntu/myapp
```

```
nano .env
```

Add variables:

```
DATABASE_URL=postgresql://user:pass@localhost:5432/mydb
```

```
PORT=8080
```

```
NODE_ENV=production
```

Secure the file

```
chmod 600 .env
```

```
chown ubuntu:ubuntu .env
```

Application loads it using dotenv or similar library

Scenario: You need to deploy a major update to your production application. As a safety measure, you want to create a complete backup of the current application directory before proceeding.

Question: How do you create a compressed backup efficiently?

Answer:

One liner answer I will use `tar -czvf`

Create backup directory

```
mkdir -p /home/ubuntu/backups
```

Create compressed archive with timestamp


```
tar -czvf /home/ubuntu/backups/app-backup-$(date +%Y%m%d-%H%M%S).tar.gz
/var/www/myapp

# Exclude node_modules and logs to reduce size
tar -czvf /home/ubuntu/backups/app-backup-$(date +%Y%m%d-%H%M%S).tar.gz \
--exclude='node_modules' \
--exclude='*.log' \
/var/www/myapp

# Verify archive created successfully
ls -lh /home/ubuntu/backups/

# Test extraction (in temp location)
mkdir /tmp/restore-test
tar -xzf /home/ubuntu/backups/app-backup-20251119-224500.tar.gz -C /tmp/restore-test

# If deployment fails, restore from backup
tar -xzf /home/ubuntu/backups/app-backup-20251119-224500.tar.gz -C /

# Keep only last 7 days of backups
find /home/ubuntu/backups/ -name "app-backup-*.tar.gz" -mtime +7 -delete
```

Scenario: Your manager asks you to set up automated database backups that run every day at 2 AM and clean up backups older than 30 days.

Question: How do you schedule this automation ?

Answer:

One liner , I will create backup script and add it in crontab

1. Create backup script

```
mkdir -p /home/ubuntu/scripts
nano /home/ubuntu/scripts/db-backup.sh
```

2. Add script content

```
#!/bin/bash
pg_dump -U postgres myapp_db | gzip > /home/ubuntu/backups/database/db-$(date +%F-%H%M).sql.gz
find /home/ubuntu/backups/database -name "*.gz" -mtime +30 -delete
```

3. Make script executable

```
chmod +x /home/ubuntu/scripts/db-backup.sh
```

4. Test backup manually

```
/home/ubuntu/scripts/db-backup.sh
```

5. Schedule cron (daily 2 AM)

```
crontab -e
```

```
0 2 * * * /home/ubuntu/scripts/db-backup.sh
```

6. Verify cron job

```
crontab -l
```

7. Check cron service

```
sudo systemctl status cron
```

8. View cron execution logs

```
grep CRON /var/log/syslog | tail -20
```

Scenario: You're trying to install Docker on a new server but getting permission denied errors. You're logged in as a regular user.

Question: How do you properly use sudo to perform system-level installations?

Answer:

```
bash
```

```
# Trying without sudo fails
```

```
apt update
```

```
# Output: E: Could not open lock file - open (13: Permission denied)
```

```
# Use sudo for system operations
```

```
sudo apt update
```

```
sudo apt install docker.io -y
```

```
# Verify installation
```

```
docker --version
```

```
# Still get permission error
```

```
# Switch to root temporarily for multiple commands
```

```
sudo su -
```

```
# Now you're root, run multiple commands
```

```
apt update
```

```
apt install docker.io docker-compose -y
```

```
systemctl start docker
```

```
systemctl enable docker
```

```
# Exit root shell
```

```
exit
```

```
# Add your user to docker group (from earlier scenario)
```

```
sudo usermod -aG docker $USER
```

```
# Best practice: Don't stay in root shell
```

```
# Use sudo for individual commands instead
```

```
sudo systemctl restart docker
```

Scenario: Your application's configuration file /etc/myapp/config.yaml has a wrong database host. You need to quickly edit it using vim on a remote server without a GUI.
Question: How do you edit the file ?

Answer:

Since there is no GUI on the server , I will be using vim or vi editor.

bash

Open file with vim

sudo vim /etc/myapp/config.yaml

Vim opened - you're in NORMAL mode

Navigate to the line with database host using arrow keys

Press 'i' to enter INSERT mode

Now you can edit like a normal editor

Change:

database_host: old-db.example.com

To:

database_host: new-db.example.com

Press ESC to exit insert mode (back to normal mode)

Save and quit:

Type: :wq and press ENTER

Quit without saving (if you made a mistake):

Type: :q! and press ENTER

Alternative using nano (easier for beginners)

sudo nano /etc/myapp/config.yaml

Edit the file

Press Ctrl+O to save

Press Ctrl+X to exit

Verify the change

cat /etc/myapp/config.yaml | grep database_host

Scenario: Your monitoring system reports that your API health check endpoint is down. You need to test it manually from the server to see what's happening.
Question: How do you test and debug the API endpoint?

Answer:

bash

Basic health check

curl http://localhost:3000/health

If it fails with "Connection refused"

Check if service is running

`sudo systemctl status myapp`

If service is running, check which port it's listening on

`netstat -tulnp | grep node`

Test with verbose output to see full response

`curl -v http://localhost:3000/health`

Which Ubuntu OS versions are you using in your company ?

Ubuntu is 25.10, codenamed Quokka

Scenario: You're trying to install a package but getting errors about incompatible OS version. You need to verify what OS and version you're running.

Question: How do you check system information?

Answer:

`bash`

Check OS and kernel version

`uname -a`

Output: Linux ip-172-31-1-10 5.15.0-1045-aws x86_64 GNU/Linux

Check distribution details

`lsb_release -a`

Output:

Distributor ID: Ubuntu

Description: Ubuntu 22.04.3 LTS

Release: 22.04

Codename: jammy

Check hostname and system info

`hostnamectl`

Shows hostname, OS, kernel, architecture

Check if 32-bit or 64-bit

`uname -m`

x86_64 (64-bit) or i686 (32-bit)

Check CPU information

`lscpu`

Check memory

`free -h`

This helps determine correct package to download

Example: Need to download Docker for Ubuntu 22.04 (jammy)

Scenario: Your application's data volume is full. Your cloud team attached a new 100GB EBS volume to your EC2 instance. You need to mount it and move data there.

Question: How do you mount the new volume and use it?

Answer:

bash

List available disks

lsblk

Shows /dev/xvdf (new volume, not mounted)

Check if volume has a filesystem

sudo file -s /dev/xvdf

Output: data (no filesystem)

Create filesystem on new volume

sudo mkfs -t ext4 /dev/xvdf

Create mount point

sudo mkdir -p /data/app-storage

Mount the volume

sudo mount /dev/xvdf /data/app-storage

Verify it's mounted

df -h | grep /data/app-storage

Set proper permissions

sudo chown -R ubuntu:ubuntu /data/app-storage

Make mount permanent (survives reboot)

Get UUID of the volume

sudo blkid /dev/xvdf

Output: UUID="abc123..."

Edit fstab

sudo nano /etc/fstab

Add this line:

UUID=abc123-your-uuid-here /data/app-storage ext4 defaults,nofail 0 2

Test fstab before rebooting

sudo umount /data/app-storage

sudo mount -a

Verify

df -h | grep /data/app-storage

Scenario: Your high-traffic application crashes with error EMFILE: too many open files. The application handles thousands of concurrent connections and is hitting system limits.

Question: How do you increase the file descriptor limit?

Answer:

bash

Check current limits for your user

ulimit -n

Output: 1024 (too low for high-traffic apps)

Check hard limit

ulimit -Hn

Output: 4096

Temporary increase (current session only)

ulimit -n 65535

Permanent fix: Edit limits configuration

sudo nano /etc/security/limits.conf

Add these lines at the end:

```
*          soft  nofile      65535
```

```
*          hard  nofile      65535
```

```
ubuntu     soft  nofile      65535
```

```
ubuntu     hard  nofile      65535
```

For systemd services, edit service file

sudo nano /etc/systemd/system/myapp.service

Add under [Service] section:

LimitNOFILE=65535

Reload systemd

sudo systemctl daemon-reload

Restart service

sudo systemctl restart myapp

Verify new limits

cat /proc/\$(pgrep -f myapp)/limits | grep "open files"

Also check system-wide file limits

cat /proc/sys/fs/file-max

Increase system-wide limit if needed

sudo sysctl -w fs.file-max=200000

```
# Make it permanent
echo "fs.file-max=200000" | sudo tee -a /etc/sysctl.conf
```

Scenario: At 2 AM, you receive a PagerDuty alert: "Production API Down - 503 Service Unavailable". Users cannot access the application. You need to diagnose and fix it quickly.
Question: What's your step-by-step troubleshooting approach?

Answer:

- **Login to the server** to confirm what's happening.
 - **Check if the API service is running** → it is failed.
 - **Look at recent service logs** → shows DB connection errors.
 - **Test database connectivity** → connection refused.
 - **Check database service** → database is not running.
 - **Try restarting the database** → fails again.
 - **Check database logs** → shows "No space left on device."
 - **Check disk usage** → disk is 100% full.
 - **Quick fix:** delete old backups to free space.
 - **Start the database again** → now it runs.
 - **Start the API application** → it also runs now.
 - **Check health endpoints** locally and externally → system is healthy.
 - **Document the incident** for audit and learning.
 - **Permanent fix:** set up disk cleanup or backup rotation so it doesn't happen again.
-

Step 1: SSH into the server

```
ssh -i prod-key.pem ubuntu@prod-api-server
```

Step 2: Check if application service is running

```
sudo systemctl status api-app
```

Service is "failed"

Step 3: Check recent logs

```
sudo journalctl -u api-app -n 50 --no-pager
```

Shows: "ECONNREFUSED: Connection refused to database"

Step 4: Check database connectivity

```
telnet db-server 5432
```

Connection refused

Step 5: Check if database service is running

```
sudo systemctl status postgresql
```

Service is dead

Step 6: Try to start database

```
sudo systemctl start postgresql
```

Fails with error

Step 7: Check why database won't start

```
sudo journalctl -u postgresql -n 100
```

Shows: "No space left on device"

Step 8: Check disk space

```
df -h
```

/var/lib/postgresql is 100% full

Step 9: Free up space quickly

```
cd /var/lib/postgresql/backups
```

```
ls -lh
```

Find old backup files

```
sudo rm backup_2025-10-*.tar.gz
```

Freed 15GB

Step 10: Start database

```
sudo systemctl start postgresql
```

```
sudo systemctl status postgresql
```

Running ✓

Step 11: Start application

```
sudo systemctl start api-app
```

```
sudo systemctl status api-app
```

Running ✓

Step 12: Verify application health

```
curl http://localhost:8080/health
```

Returns 200 OK ✓

Step 13: Check from external monitoring

```
curl https://api.yourcompany.com/health
```

Working ✓

Step 14: Document incident

```
echo "Incident: API down due to disk full. Removed old backups. Need to implement backup rotation." >> /var/log/incidents.log
```

Step 15: Implement permanent fix (schedule cleanup cron job)

Which Linux distributions have you used and when would you choose each?

Ubuntu/Debian

- Uses apt package manager
- Best for: Cloud deployments, DevOps toolchains, containers
- Faster updates, large community support

RHEL/CentOS/Rocky

- Uses yum/dnf package manager
- Best for: Enterprise environments requiring long-term support
- Stricter stability, SELinux enabled by default

Amazon Linux 2

- Optimized for AWS
- Pre-configured with AWS tools

Key differences: Package manager, stability vs freshness, licensing, file locations, SELinux presence.

How do you securely connect to remote Linux servers?

SSH with keys (recommended)

bash

ssh -i ~/.ssh/id_rsa user@server

Benefits: Secure, passwordless, can disable password auth, audit trail.

Setup process:

bash

Generate key

ssh-keygen -t rsa -b 4096 -C "your_email@example.com"

Copy to server

ssh-copy-id user@server

Disable password auth in /etc/ssh/sshd_config

PasswordAuthentication no

What are your top 10 essential Linux commands

bash

ls -lah *# list files with details*

cd /path *# change directory*

grep -r "text" *# search recursively*

tail -f *# follow logs in real-time*

journalctl -u *# view service logs*

ps aux *# see all processes*

htop/top *# live resource monitoring*

df -h *# disk usage by filesystem*

du -sh * *# folder sizes*

ss -tulpn *# processes listening on ports*

SCENARIO: SSH connection suddenly drops after 2 minutes. How do you fix it?

Diagnosis:

- Check if timeout is client or server side
- Review /var/log/auth.log or /var/log/secure

Solution:

bash

Client side (~/.ssh/config or /etc/ssh/ssh_config)

ServerAliveInterval 60

ServerAliveCountMax 3

Server side (/etc/ssh/sshd_config)

ClientAliveInterval 60

ClientAliveCountMax 3

Restart SSH

systemctl restart sshd

Explain Linux directory structure

/etc → system configuration files

/var → variable data (logs, cache, mail)

/home → user home directories

/root → root user home directory

/tmp → temporary files (cleared on reboot)

/opt → optional/third-party software

/usr → user programs and utilities

/bin → essential user binaries

/sbin → system administration binaries

/dev → device files

/proc → process and kernel information

/sys → kernel and system information

SCENARIO: Your EC2 instance rebooted and your application can't find data at /data. The mount was working before reboot. How do you diagnose and fix this permanently?

Diagnosis Steps:

bash

1. Check if volume is attached

lsblk

2. Check current mounts

mount | grep data

df -h

3. Check /etc/fstab

cat /etc/fstab

4. Try manual mount

mount -a

5. Check for errors

dmesg | grep -i error

journalctl -xe

Solution:

bash

Find UUID of device

```
blkid /dev/xvdf
```

```
# Add to /etc/fstab (use UUID, not device name!)
```

```
UUID=xxxx-xxxx /data ext4 defaults,nofail 0 2
```

```
# Test fstab without rebooting
```

```
mount -a
```

```
# Verify
```

```
df -h | grep data
```

Why UUID? Device names (/dev/xvdf) can change; UUIDs are permanent.

SCENARIO: "No space left on device" error but df -h shows 40% space available. What's happening?

One line answer: Inode exhaustion (too many small files).

Step 1: The Decision Check

Run this command to see which problem you actually have:

Bash

```
df -i
```

Scenario A: If IUse% is 100% (Inode Exhaustion)

The Problem: You have disk space (pages in the book), but no "Table of Contents" slots left. This happens due to millions of tiny files (like sessions or cache).

1. Find the bad folder (Fastest method): Do not use find on the whole disk. Use du:

Bash

```
du --inodes -d 1 / | sort -rn | head -n 10
```

(Repeat this command inside the largest directory found until you hit the bottom).

2. The Fix: Once you find the folder (e.g., /var/lib/php/sessions), clear it:

Bash

```
# Efficiently delete files older than 24 hours
```

```
find /var/lib/php/sessions -type f -mtime +1 -delete
```

Scenario B: If IUse% is low (Phantom Files)

The Problem: You deleted a huge log file, but a program (like Apache/Nginx) is still holding it open, so the space wasn't released.

1. Find the culprit:

Bash

```
ls -l | grep '(deleted)'
```

Look for large files in the output.

2. The Fix: Restart the service listed in the output to release the space.

Bash

```
systemctl restart nginx # (Or apache2, mysqld, etc.)
```

SCENARIO: A user complains they can't write to /opt/application/config even though they own the file. Why?

The Core Issue: "Ownership" is not enough

Just because a user owns a file (user:user) does not guarantee they can edit it. The operating system checks multiple layers of permission (File Attributes, Parent Directory, Filesystem, and Security Modules) before allowing a write operation.

Phase 1: The "Editor Trap" (Most Likely Cause)

If the user is trying to save the file using a text editor (like **Vim, Nano, or VS Code**), they are likely failing because they lack permissions on the **Parent Directory**.

- **Why:** Modern editors do "Atomic Saving." They don't just write inside the file; they create a temporary file (e.g., .config.swp), delete the old file, and rename the temp file to the original name.
- **The Rule:** To delete or rename a file, you need **Write (+w) and Execute (+x)** permissions on the **Directory**, not just the file.

Diagnosis:

Check directory permissions

ls -ld /opt/application

Solution: Give the user write access to the directory, or add them to the directory's group.

Bash

Example: Give group write access

chmod g+wx /opt/application

Phase 2: File Attributes (The "Hidden" Lock)

Linux has "extended attributes" that override standard read/write permissions. The most common are **Immutable** (cannot be changed at all) and **Append Only** (can add data, but cannot overwrite/delete).

Diagnosis:

Bash

lsattr /opt/application/config

- Look for an **i** (Immutable) or an **a** (Append Only) in the output (e.g., ----i-----).

Solution: You need root access to remove these attributes.

Bash

sudo chattr -i /opt/application/config # Remove Immutable

sudo chattr -a /opt/application/config # Remove Append Only

Phase 3: Access Control Lists (ACLs)

If standard permissions look correct but access is still denied, look for a **+** symbol at the end of the permission string (e.g., -rw-rw-r--+). This indicates ACLs are active and might be explicitly denying this specific user.

Diagnosis:

Bash

getfacl /opt/application/config

- Look for lines like **user:username:r--** which might override the standard owner permissions.

Solution: Adjust the ACL to grant write access.

Bash
setfacl -m u:username:rw /opt/application/config

Phase 4: Security Modules (SELinux / AppArmor)

If permissions and attributes are correct, the kernel's security module is likely blocking the process (e.g., httpd or java) from writing to that specific path.

Diagnosis:

1. **Check status:** Is SELinux Enforcing?

Bash
sestatus

2. **Check the logs:** This is the only way to be sure.

Bash
grep "denied" /var/log/audit/audit.log | grep "config"

Solution: Restore the correct security context for that file path.

Bash
restorecon -v /opt/application/config

Phase 5: Filesystem Issues

Finally, check the physical state of the storage.

1. **Read-Only Mount:** The disk might have flipped to Read-Only due to an error.

Bash
mount | grep "ro,"

2. **Disk Full:**

Bash
df -h /opt # Check space
df -i /opt # Check inodes (file count limits)

Summary Checklist

If a user asks "Why can't I write?", follow this order:

1. **Check Attributes:** lsattr file (Is it immutable?)
2. **Check Directory:** ls -ld . (Can the editor create temp files?)
3. **Check ACLs:** getfacl file (Is there a hidden restriction?)
4. **Check SELinux:** grep denied /var/log/audit/audit.log

Compare filesystems: EXT4, XFS, Btrfs

Filesystem	Best Use Case	Pros	Cons
EXT4	General purpose, default	Stable, mature, good performance	Limited scalability
XFS	Large files, databases	High performance, parallel I/O	Difficult to shrink
Btrfs	Snapshots, RAID	Copy-on-write, compression, snapshots	Less mature, some stability concerns
ZFS	Enterprise storage	Snapshots, compression, data integrity	High memory usage, licensing

List and manage running processes

View processes

```
ps aux          # all processes
ps aux | grep nginx # specific process
pgrep nginx     # get PID only
pgrep -a nginx  # with full command
```

Process tree

```
pstree
ps auxf
```

Detailed process info

```
top
htop          # better interface
```

SCENARIO: Kill a process, but which signal?

```
bash
# Graceful shutdown (default)
kill PID      # sends SIGTERM (15)
```

Allow cleanup

```
kill -15 PID
systemctl stop service
```

Force kill (last resort)

```
kill -9 PID    # sends SIGKILL
killall -9 process_name
```

Other useful signals

```
kill -HUP PID  # reload config (SIGHUP)
kill -USR1 PID # application-specific
```

Best practice: Always try graceful shutdown first, allow 10-30 seconds, then force kill.

Check CPU and memory usage

```
bash
# Real-time monitoring
top
htop
```

Memory summary

```
free -h
```

Detailed memory

```
cat /proc/meminfo
```

CPU statistics

```
mpstat 1
```

```
sar -u 1 3  
vmstat 2
```

```
# Per-process memory  
ps aux --sort=-%mem | head -10  
ps aux --sort=-%cpu | head -10
```

```
# I/O statistics  
iostat -x 2 3  
iotop
```

SCENARIO: After deploying a new version, systemctl restart nginx hangs forever. Your website is down. Walk through troubleshooting.

Step-by-step approach:

```
bash
```

```
# 1. Check current status  
systemctl status nginx
```

```
# 2. Check if old process is stuck  
ps aux | grep nginx
```

```
# 3. Review recent logs  
journalctl -u nginx -n 100 --no-pager  
tail -50 /var/log/nginx/error.log
```

```
# 4. Test configuration  
nginx -t
```

```
# 5. Check port conflicts  
ss -tulpn | grep :80  
ss -tulpn | grep :443
```

```
# 6. Check for file locks  
lsof | grep nginx
```

```
# 7. If stuck, force kill  
systemctl kill --signal=KILL nginx  
# Or  
pkill -9 nginx
```

```
# 8. Check what's holding the port  
kill -9 $(lsof -ti:80)
```

```
# 9. Start fresh  
systemctl start nginx
```

10. Verify

curl -I localhost

systemctl status nginx

Root cause analysis:

- Configuration syntax error → nginx -t shows it
- Port already in use → another nginx or apache running
- Permission issue → check error logs
- File descriptor limit → check ulimit -n

Systemd service management essentials

bash

Service operations

systemctl start nginx

systemctl stop nginx

systemctl restart nginx

systemctl reload nginx *# reload config without restart*

systemctl status nginx

Enable/disable on boot

systemctl enable nginx

systemctl disable nginx

View logs

journalctl -u nginx

journalctl -u nginx -f *# follow*

journalctl -u nginx --since "1 hour ago"

journalctl -u nginx --since today

journalctl -xe *# recent errors*

List services

systemctl list-units --type=service

systemctl list-units --state=failed

Runlevels (targets)

systemctl get-default

systemctl set-default multi-user.target

systemctl isolate rescue.target

SCENARIO: Service shows "active (running)" but application doesn't respond. How do you troubleshoot?

Common root causes

- App started but listening only on **127.0.0.1** instead of 0.0.0.0.
- **Firewall** (ufw/iptables/security groups) blocking the port.
- App waiting on a **dependency** that is down (DB/Redis).
- **Out of file descriptors** → cannot accept new connections.
- Using the **wrong config file**, so app binds to wrong port or fails silently.

— Simple Troubleshooting Steps I will follow

- **Check service status** first to confirm systemd thinks it's running.
- **Check if the process actually exists** → sometimes service is "running" but the process died.
- **Check if the app is listening on expected ports** (e.g., 8080).
- **Test access locally** using curl or telnet to see if the app responds internally.
- **Check application logs** for errors, crashes, config issues, or startup failures.
- **Filter only error logs** to quickly spot critical problems.
- **Check resource limits** (memory, open files, CPU) that may be blocking the app.
- **Check dependent services** like database, Redis, or message queues.
- **Try running the app manually** to confirm if configuration or startup is broken.

1. Verify service status

```
systemctl status myapp
```

2. Check if process is actually running

```
ps aux | grep myapp  
pgrep -a myapp
```

3. Check listening ports

```
ss -tulpn | grep myapp  
netstat -tulpn | grep 8080
```

4. Test connectivity

```
curl -v localhost:8080  
telnet localhost 8080
```

5. Check application logs

```
journalctl -u myapp -n 100  
tail -f /var/log/myapp/application.log
```

6. Check for errors

```
journalctl -u myapp -p err
```

7. Check resource constraints

```
cat /proc/$(pgrep myapp)/limits  
systemctl show myapp | grep -i limit
```

8. Check dependencies

```
systemctl list-dependencies myapp
```

9. Test application directly

```
/usr/bin/myapp --config /etc/myapp/config.yml
```

Common causes:

- Application bound to 127.0.0.1 instead of 0.0.0.0
- Firewall blocking port
- Application waiting for dependency (database, Redis)

- Out of file descriptors
- Wrong configuration file loaded

SCENARIO: Server has load average of 15.0 on a 4-core machine at 2 AM. Customers complaining of slow response. What are your first actions?

Immediate triage

- **Check system overview** (uptime, load, CPU, memory) to confirm the spike.
- **Identify top CPU-using processes** to see who is causing the load.
- **Check I/O wait** to find out if the problem is CPU or disk-related.
- **Check available memory & swap** to detect memory pressure.
- **Check active network connections** to see if you are being flooded.
- **Check system logs** for errors, OOM kills, disk failures, or crashes.

Analysis & quick actions

If CPU-bound

- Check which **threads** inside the app are consuming CPU.
- Temporarily **renice** the culprit process to reduce priority.
- Restart the application if it is stuck or spinning.

If Disk / I/O-bound

- Use iotop and iostat to find which process is hammering the disk.
- Check kernel logs and smartctl for failing disk or read/write errors.

If Network overload / DDoS

- Count connections per IP to detect abnormal traffic.
- Block the malicious IP temporarily using firewall rules.
- Apply rate-limiting on the service port to stabilize the server.

Follow-up permanent actions

- Add more servers / autoscaling to handle traffic bursts.
- Review app logs for performance issues or errors.
- Check if any new deployment caused the spike.
- Improve alerting thresholds to catch issues earlier.

Commands for Immediate triage

bash

1. Quick system overview

uptime

top -bn1 | head -20

2. Identify top CPU consumers

ps aux --sort=-%cpu | head -10

3. Check I/O wait (is it CPU or disk?)

iostat -x 2 3

High %iowait means disk bottleneck

4. Check memory

free -h

If swap is high, memory pressure

5. Check active connections

ss -s

netstat -an | grep ESTABLISHED | wc -l

6. Check for errors

journalctl -p err -n 50

dmesg | tail -20

Analysis & Action:

If CPU bound:

bash

Find the culprit

top -H -p \$(pgrep -d',' myapp) *# threads view*

Temporary mitigation

renice -n 10 -p PID *# lower priority*

systemctl restart myapp *# if stuck*

If I/O bound:

bash

iostat *# see which process*

iostat -x 2 *# disk stats*

Check for disk issues

dmesg | grep -i error

smartctl -a /dev/sda *# disk health*

If network/connection flood:

bash

Check connections per IP

netstat -ntu | awk '{print \$5}' | cut -d: -f1 | sort | uniq -c | sort -n

Block if DDoS

iptables -A INPUT -s 1.2.3.4 -j DROP

Rate limit

iptables -A INPUT -p tcp --dport 80 -m limit --limit 25/minute --limit-burst 100 -j ACCEPT

Follow-up actions:

- Scale horizontally (add servers)
- Check application logs for errors
- Review recent deployments
- Set up better alerting thresholds

SCENARIO: Server won't boot, drops to emergency mode. How do you recover?

Recovery steps:

1. Boot into single-user/rescue mode

At GRUB: press 'e', add to kernel line:

`systemd.unit=rescue.target`

or

`single`

2. Once in rescue mode, check what failed

`journalctl -xb`

`systemctl --failed`

3. Check /etc/fstab for bad entries

`cat /etc/fstab`

Comment out problematic mounts

4. Check filesystem corruption

`fsck /dev/sda1` *# unmount first!*

5. Remount root as read-write

`mount -o remount,rw /`

6. Fix issues, then reboot

`systemctl reboot`

What are Kernel space vs User space?

Kernel Space:

- Device drivers
- Process scheduling
- Memory management
- System calls
- Full hardware access

User Space:

- Applications
- Libraries
- Shells
- No direct hardware access

Communication: System calls (read, write, open, etc.)

Find IP and network configuration

IP addresses

`ip addr show` *# Show all IP addresses assigned to all interfaces*

`ip a` *# Short form for the same command*

Routing table

`ip route show` *# Shows how traffic is routed (default gateway, subnets)*

`route -n` *# Older command, displays routing table in numeric form*

Network interfaces

`ip link show` *# List all network interfaces and their states (UP/DOWN)*

`nmcli dev show` *# Shows interface details if using NetworkManager*

ifconfig # Deprecated, but still widely used to view interface info

DNS configuration

cat /etc/resolv.conf # Shows DNS servers being used

systemd-resolve --status # Shows DNS per interface (systemd-resolved)

Hostname

hostname # Shows the system's hostname

hostnamectl # Shows hostname + OS + kernel info, and allows setting hostname

SCENARIO: You can ping the server but can't SSH to it. Troubleshoot.

I will follow these steps to troubleshoot.

- **Check if SSH port 22 is reachable** → using telnet or nc helps confirm if the port is open.
 - **Check if SSH service is running** (from console/KVM access) to ensure sshd didn't crash.
 - **Check if SSH is listening on port 22** — sometimes it's bound to the wrong IP.
 - **Check firewall rules** (iptables/ufw/firewalld) to see if port 22 is blocked locally.
 - **Check SSH configuration** for wrong settings that prevent login.
 - **Check authentication logs** for errors (e.g., key mismatch, bans, too many attempts).
 - **Test SSH locally from the server** to confirm if the issue is remote-only.
 - **Check hosts.allow / hosts.deny** to ensure the client IP is not blocked.
-

Common Root Causes

- **Firewall or security group blocking port 22**
 - **SSH service not running**
 - **Server hit MaxStartups (too many connections)**
 - **SSH bound to 127.0.0.1 instead of 0.0.0.0**
 - **Cloud provider rules (AWS/GCP/Azure) blocking the port**
 - **fail2ban or intrusion prevention banned your IP**
-

Commands to use

1. Test SSH port

telnet server 22

nc -zv server 22

2. Check if SSH is running on server (if you have console access)

systemctl status sshd

ps aux | grep sshd

3. Check if SSH is listening

ss -tulpn | grep :22

4. Check firewall

iptables -L -n | grep 22

firewall-cmd --list-all

ufw status

5. Check SSH config

cat /etc/ssh/sshd_config | grep -v "^#"

6. Check logs

tail -50 /var/log/auth.log

journalctl -u sshd

7. Test from localhost

ssh localhost

8. Check hosts.allow/hosts.deny

cat /etc/hosts.allow

cat /etc/hosts.deny

Check which process is using a port

bash

Modern method

ss -tulpn | grep :8080

Legacy method

netstat -tulpn | grep :8080

Find process by port

lsof -i :8080

fuser -n tcp 8080

Kill process on port

kill \$(lsof -ti:8080)

fuser -k 8080/tcp

SCENARIO: New microservice on port 8080, load balancer health checks timing out. Application logs show it's running. Fix the connectivity.

I will troubleshoot in following way:

- **Test the service locally** using curl to ensure the app is responding internally.
- **Check if the service is actually listening on port 8080.**
- **Verify what IP/interface the app is bound to** — if it's bound to 127.0.0.1, external traffic cannot reach it.
- **Fix the app config** by binding to 0.0.0.0:8080 so it accepts external connections.
- **Check firewall rules** to see if port 8080 is blocked on the OS firewall.
- **Add or update firewall rules** to allow access to port 8080.
- **Test connectivity from another server** using the private IP to ensure the port is reachable over the network.

- **Check cloud security groups** (AWS/GCP/Azure) to confirm port 8080 is allowed.
- **Verify load balancer settings** — ensure the health check path (e.g., /health) is correct.
- **Watch application logs** to confirm it's receiving and responding to health check requests.

Common Root Cause

App is listening on 127.0.0.1, not 0.0.0.0.

This is the #1 reason new services fail behind a load balancer.

Commands to use

1. Test locally first

```
curl localhost:8080/health
```

```
curl 127.0.0.1:8080/health
```

2. Check if service is listening

```
ss -tulpn | grep 8080
```

3. Check what interface it's bound to

```
netstat -tulpn | grep 8080
```

If shows 127.0.0.1:8080 → bound only to localhost!

4. Check application config

Fix: bind to 0.0.0.0:8080 instead of 127.0.0.1:8080

5. Check firewall

```
iptables -L -n | grep 8080
```

```
firewall-cmd --list-all
```

6. Add firewall rule if needed

```
firewall-cmd --add-port=8080/tcp --permanent
```

```
firewall-cmd --reload
```

Or iptables

```
iptables -A INPUT -p tcp --dport 8080 -j ACCEPT
```

Or ufw

```
ufw allow 8080/tcp
```

7. Test from another server

```
curl -v http://private-ip:8080/health
```

8. Check cloud security groups (AWS/Azure/GCP)

Ensure port 8080 is open in security group

9. Verify load balancer configuration

Check target group health check path matches /health

10. Check application logs
journalctl -u myapp -f

How will you do DNS troubleshooting?

I will troubleshoot in following way:

- **Check if DNS resolution works** using dig, nslookup, or host to confirm the issue.
- **Check the system's DNS configuration** (/etc/resolv.conf and /etc/hosts) to ensure correct nameservers.
- **Trace the full DNS resolution path** with dig +trace to find where it fails (root, TLD, or authoritative server).
- **Query a specific nameserver directly** (e.g., @8.8.8.8) to see if the problem is local or upstream.
- **Perform reverse DNS lookups** to check PTR records for IP addresses. ((**Forward DNS (normal lookup)**: A record: api.example.com → 203.0.113.10
Reverse DNS (PTR record): PTR record: 203.0.113.10 → api.example.com)
- **Flush DNS cache** to clear stale or corrupted entries.

✓ Most common causes

- Wrong or missing **nameserver** in /etc/resolv.conf
- Local network DNS server unreachable
- Firewall blocking UDP/TCP **port 53**
- Issues at upstream DNS provider
- Broken / stale DNS cache
- Wrong entry in /etc/hosts overriding domain
- Local DNS resolver (systemd-resolved) malfunctioning

Commands

Check DNS resolution

dig google.com

nslookup google.com

host google.com

Check DNS configuration

cat /etc/resolv.conf

cat /etc/hosts

Trace DNS path

dig +trace google.com

Check specific nameserver

dig @8.8.8.8 google.com

Reverse DNS lookup

dig -x 8.8.8.8


```
# Flush DNS cache
systemd-resolve --flush-caches
# Or
resolvectl flush-caches
```

How do you manage users and groups

```
bash
# Add user
useradd -m -s /bin/bash john
passwd john

# Add user with specific UID/GID
useradd -u 1500 -g developers john

# Modify user
usermod -aG sudo john      # add to sudo group
usermod -s /bin/zsh john   # change shell
usermod -L john            # lock account
usermod -U john            # unlock account

# Delete user
userdel john               # keep home dir
userdel -r john            # remove home dir

# Group management
groupadd developers
groupdel developers
gpasswd -a john developers # add user to group
gpasswd -d john developers # remove from group

# View user info
id john
groups john
finger john
getent passwd john
```

Your developer Satish needs to restart Nginx during deployments, but security policy says he must not get full sudo access.

How will you give him permission to run only:

**systemctl restart nginx
and ensure he doesn't need a password?**

- **One liner : add him to /etc/sudoers file)**

- Use visudo to edit safely.
- Add a NOPASSWD rule for the specific command.

```
sudo visudo
```

satish ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart nginx

- Verify using:

sudo -l -U satish (lowercase L)

SCENARIO: Security audit requires rotating all SSH keys. You have 50 servers and 10 engineers. How do you implement without locking anyone out?

Strategy:

1. Avoid Big Bang — Use Overlap Method

Always add new keys first → verify access → then remove old keys; this prevents mass lockouts.

2. Phase 1 — Add New Keys (Both Keys Work)

Generate new Ed25519 keys and append them to `authorized_keys` using Ansible or a safe script.

3. Phase 2 — Verify New Keys

Engineers test login forcing the new key via `~/.ssh/config`; old keys remain as fallback.

4. Phase 3 — Remove Old Keys

After everyone confirms access, remove old key entries via Ansible and audit `authorized_keys`.

5. Long-Term Fix — Use SSH Certificates

Set up an SSH CA so keys expire automatically (e.g., 24 hours), eliminating manual key rotation.

This is a classic high-stakes operations task. The biggest mistake people make is the **"Big Bang" approach** (wiping old keys and adding new ones simultaneously), which guarantees lockouts if a copy fails.

The correct, zero-downtime strategy is the **"Overlap Method"** (Add → Verify → Remove).

Here is the step-by-step implementation plan for 50 servers.

Phase 1: The Setup (The "Overlap" State)

You must create a period where **BOTH** the old and new keys work.

1. Generate New Keys

Ask all 10 engineers to generate new keys locally.

Recommendation: Use Ed25519 (faster and more secure than RSA).

Bash

```
ssh-keygen -t ed25519 -C "email@company.com-2025-rotation"
```

2. Deploy New Keys (Append Only)

Do not edit files manually. Use a configuration manager (Ansible is best for 50 servers) to append the new keys to `~/.ssh/authorized_keys.1`

If using Ansible:

YAML

```
- name: Add new SSH keys (Append)
  authorized_key:
    user: satish
    state: present
    key: "{{ item }}"
  with_file:
    - public_keys/satish_new.pub
```

If using Bash (fallback):

Bash

```
cat satish_new.pub | ssh user@server "cat >> ~/.ssh/authorized_keys"
```

Phase 2: The Verification (The Human Check)

Now that the servers accept both keys, you must force engineers to test the *new* key explicitly.

Instruction to Engineers:

"Please configure your ~/.ssh/config to strictly use the new key for testing."

Plaintext

Host my-server-test

HostName 192.168.1.50

IdentityFile ~/.ssh/id_ed25519_new

IdentitiesOnly yes

- **Success:** Engineer confirms they can log in.
- **Failure:** They can still log in with the old key to debug the issue. **Zero lockout.**

Phase 3: The Cleanup (Revocation)

Only once all 10 engineers confirm access via new keys do you remove the old ones.

1. Remove Old Keys

Update your Ansible playbook or script to explicitly remove the old public key strings.

Ansible:

YAML

```
- name: Remove OLD SSH keys
  authorized_key:
    user: satish
    state: absent
    key: "{{ item }}"
  with_file:
    - public_keys/satish_OLD.pub
```

2. Audit the File

Run a quick check on a few servers to ensure only 1 key exists per user.

Bash

```
awk '{print $NF}' ~/.ssh/authorized_keys | sort | uniq -c
```

Advanced: The "Forever Fix" (SSH Certificates)

If you do this audit frequently, **stop using static SSH keys.**

Implement an **SSH Certificate Authority (CA).**

1. You (the CA) sign the engineer's key with an expiration (e.g., valid for 24 hours).

2. Server trusts the CA, not the specific key.
3. **Rotation is automatic:** Keys expire daily; engineers must re-authenticate (via SSO/MFA) to get a new signed certificate.

How do you Secure SSH configuration?

- **Disable root login** so attackers cannot directly target the root account.
- **Disable password authentication** and only allow SSH keys for safer, stronger access.
- **Use strong modern ciphers** to prevent weak encryption vulnerabilities.
- **Limit which users and groups can SSH** to minimize access surface.
- **Change the default SSH port** to reduce automated bot attacks (not a primary defense, but helpful).
- **Set idle timeouts** to disconnect inactive sessions and reduce risk.
- **Restart SSH safely** after changes to apply the updated configuration.
- **Install fail2ban** to automatically block repeated failed login attempts.

Commands to use

Edit /etc/ssh/sshd_config

Disable root login

PermitRootLogin no

Disable password authentication

PasswordAuthentication no

ChallengeResponseAuthentication no

Use strong ciphers only

Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com

Limit users

AllowUsers john jane

AllowGroups sshusers

Change default port (security through obscurity)

Port 2222

Set idle timeout

ClientAliveInterval 300

ClientAliveCountMax 2

Restart SSH

systemctl restart sshd

Install fail2ban

```
apt install fail2ban
systemctl enable fail2ban
```

How do you find files owned by a user

Find all files owned by user

```
find / -user olduser 2>/dev/null
```

Change ownership to new user

```
find / -user olduser -exec chown newuser {} \; 2>/dev/null
```

Find and list by type

```
find /home -user olduser -type f # files only
```

```
find /home -user olduser -type d # directories only
```

Find by group

```
find / -group oldgroup 2>/dev/null
```

Find files with specific permissions

```
find / -perm 777 2>/dev/null
```

```
find / -perm /4000 2>/dev/null # SUID files
```

How do you install, update, remove packages

Debian/Ubuntu (apt):

```
bash
```

Update package list

```
apt update
```

Upgrade packages

```
apt upgrade # safe upgrade
```

```
apt full-upgrade # handle dependencies
```

Install

```
apt install nginx
```

Remove

```
apt remove nginx # keep config
```

```
apt purge nginx # remove config too
```

Search

```
apt search nginx
```

```
apt-cache search nginx
```

Show package info

```
apt show nginx
```

List installed

```
apt list --installed
```

Clean cache

```
apt autoclean
```

```
apt autoremove
```

SCENARIO: You need to install a specific version of a package. How?

Debian/Ubuntu:

```
bash
```

List available versions

```
apt-cache policy nginx
```

Install specific version

```
apt install nginx=1.18.0-0ubuntu1
```

Hold version (prevent updates)

```
apt-mark hold nginx
```

Unhold

```
apt-mark unhold nginx
```

How do you manage system patches and updates

Best practices:

- Test in staging first
- Schedule during maintenance windows
- Take snapshots/AMI before major updates
- Use Ansible for consistency across fleet
- Monitor logs after updates

1. Regular Updates Through Automation:

I automate OS updates using tools like **Ansible**, **AWS SSM Patch Manager**, or **cron-based unattended-upgrades** to ensure security patches are applied consistently across all servers.

2. Staged Rollout (Safe Deployment):

Updates are first applied on **test/staging servers**, validated for app compatibility, and then rolled out to production using a **batch/blue-green strategy** to avoid downtime.

3. Controlled Package Upgrades:

I use commands like `apt update && apt list --upgradable` to review changes, and `apt upgrade` or `apt-get dist-upgrade` when kernel or major updates are needed.

4. Reboot & Kernel Patch Handling:

For kernel updates, I rely on tools like **needrestart**, **kexec**, or scheduled maintenance windows to reboot safely, ensuring no impact to running workloads.

5. Monitoring & Compliance:

I use **Prometheus/Grafana**, **canonical-livepatch**, and vulnerability scanners (**Lynis/Trivy/Qualys**) to track outdated packages, security alerts, and patch compliance.

How do you Check package version and files?

```
bash
# Debian/Ubuntu
dpkg -l | grep nginx      # list installed
dpkg -L nginx             # files installed by package
dpkg -S /etc/nginx/nginx.conf # which package owns file

# RHEL/CentOS
rpm -qa | grep nginx      # list installed
rpm -ql nginx             # files in package
rpm -qf /etc/nginx/nginx.conf # which package owns file

# Check version
nginx -v
python3 --version
```

SCENARIO:

Your deployment script runs with
`bash deploy.sh`
but fails with
`./deploy.sh: command not found`
even though the file is executable. What's wrong?

Root Cause: Incorrect or Missing Shebang + Possible Windows Line Endings

Answer for Interview

"./deploy.sh fails because the script either has a missing/wrong shebang or Windows CRLF line endings. With `bash deploy.sh` the interpreter is forced manually, but when running directly, Linux can't find the correct interpreter."

Why it works with `bash deploy.sh`:

You explicitly told Bash to run the script, so it ignores the shebang and Windows line endings.

Why it fails with `./deploy.sh`:

Linux tries to run it using the interpreter defined in the **first line** (the shebang), but if:

- The shebang is **missing**,
- The shebang path is **wrong** (e.g., `#!/bin/sh^M`), or
- The file has **Windows CRLF endings** (^M)
Linux cannot find the interpreter → triggers "**command not found**".

Diagnosis Steps (3–4 essential commands)

```
file deploy.sh      # Shows if file has CRLF or incorrect format
head -1 deploy.sh   # Inspect shebang line
ls -l deploy.sh     # Verify executable bit
cat -A deploy.sh | head # If you see ^M, it has Windows line endings
```

Solution

1. Add/Correct the shebang

```
#!/bin/bash
```

2. Convert Windows line endings → Unix

```
dos2unix deploy.sh
```

or

```
sed -i 's/\r$//' deploy.sh
```

3. Verify interpreter path exists

```
which bash # should return /bin/bash
```

4. Ensure file is executable

```
chmod +x deploy.sh
```

✓ Final Interview Answer (1–2 lines):

“./deploy.sh fails because the script either has a missing/wrong shebang or Windows CRLF line endings. With bash deploy.sh the interpreter is forced manually, but when running directly, Linux can’t find the correct interpreter.”

How do you Parse logs with grep, awk, sed

bash

grep examples

```
grep "ERROR" app.log
```

```
grep -i "error" app.log # case insensitive
```

```
grep -r "ERROR" /var/log/ # recursive
```

```
grep -v "INFO" app.log # exclude
```

```
grep -A 5 "ERROR" app.log # 5 lines after
```

```
grep -B 5 "ERROR" app.log # 5 lines before
```

```
grep -C 5 "ERROR" app.log # 5 lines context
```

awk examples

```
awk '{print $1,$5}' access.log # columns 1 and 5
```

```
awk '/ERROR/ {print $0}' app.log # filter rows
```

```
awk '$6 > 500 {print $1,$6}' app.log # conditional
```

```
awk -F',' '{print $2}' data.csv # CSV delimiter
```

sed examples

```
sed 's/error/ERROR/g' app.log # replace
```

```
sed -n '1,50p' app
```

You are working on a production server and repeatedly typing `ls -alF` to check file details. It's slowing you down.

How do you create a shortcut so you can type just ll?

Answer:

Create a Bash alias in `~/.bashrc` or `~/.zshrc`:

```
alias ll="ls -alF"
```

Reload the shell:

source ~/.bashrc

Question:

You frequently use `docker ps -a` to check containers. How do you create a quick shortcut so you only type `dps`?

Answer:

Add an alias:

```
alias dps="docker ps -a"
```

Question:

You don't want to type this long SSH command every time:

```
ssh -i /home/ubuntu/sec/prod.pem ubuntu@10.0.22.123
```

How can you create a one-word SSH shortcut such as `ssh prod`?

Answer:

Use **SSH config file** (`~/.ssh/config`):

Host prod

```
    HostName 10.0.22.123
```

```
    User ubuntu
```

```
    IdentityFile /home/ubuntu/sec/prod.pem
```

Then simply:

```
ssh prod
```

You ran a command and got a "permission denied" error. You want to run the same command again with `sudo`, without retyping everything.

How do you do it?

Answer:

Use history expansion:

```
sudo !!
```

Question:

You ran a command yesterday but don't remember the exact syntax.

What shortcut allows you to search your history interactively?

Answer:

Press:

```
Ctrl + r
```

Then type part of the command to auto-search.

Question:

You previously ran `ssh prod-server` but don't remember the full command.

How can you run the **most recent command starting with "ssh"**?

Answer:

Use:

```
!ssh
```

Question:

You manage dev, staging, and prod servers. Instead of copying full SSH commands, how do you set up shortcuts?

Answer:

Add entries in ~/.ssh/config:

Host dev

HostName 10.0.x.x

User ubuntu

IdentityFile ~/.ssh/dev.pem

Host staging

HostName 10.0.x.x

User ubuntu

IdentityFile ~/.ssh/staging.pem

Host prod

HostName 10.0.x.x

User ubuntu

IdentityFile ~/.ssh/prod.pem

Now connect with:

ssh dev

ssh staging

ssh prod

A service keeps restarting (systemd Restart=always). How do you troubleshoot?**Answer:**

- Check logs via journalctl -u servicename -xe for crash loops.
- Run binary manually to see real errors (/usr/bin/app).
- Verify service file paths, permissions, and environment variables.
- Disable restart temporarily: systemctl edit servicename → override Restart=no for debugging.
- Common fix: missing config, port in use, bad permissions, wrong ExecStart.

CPU spikes every hour — how do you pinpoint the process?**Answer:**

- Use top, htop, or pidstat -u 60 1 to capture periodic spikes.
- Check cron jobs with grep -R "" /etc/cron* for hourly tasks.
- Analyze systemd timers: systemctl list-timers.
- Enable logging with sar -u 1 3600 or atop for historical CPU analysis.
- Usually culprit: backup scripts, indexing jobs, cron-based imports.

Memory leak suspected — how do you confirm and mitigate without rebooting?**Answer:**

- Use top, ps aux --sort=-rss, and smem to identify memory-hog processes.
- Watch growth over time: watch -n 5 ps -o pid,rss,command | head.
- Check kernel logs for warnings: dmesg | grep -i memory.
- Mitigation: restart the leaking service, add limits via systemd (MemoryMax=), fix app code.
- If unavoidable → increase swap temporarily: sudo swapon.

OOM-killer killed your process — how do you find why and prevent it next time?

Answer:

- Check `dmesg | grep -i kill` or `/var/log/syslog` to see which process was killed and its memory usage.
- Verify memory pressure: `free -h`, `vmstat 1`, `ps --sort=-rss`.
- Set systemd limits (`MemoryMax` or `LimitRSS`), container limits, or optimize the app.
- Add swap or resize instance if consistently hitting memory ceiling.
- Use monitoring dashboards to track memory trends before OOM hits.